DOCUMENTACIÓN DEL CÓDIGO FUENTE

# DocMerge
# Guía técnica

Arquitectura, punto de entrada y flujo de ejecución.
Ordenación y transferencia de tablas DOCX mediante listados PDF.

Versión de producto 2.1.0

**Fecha de revisión:** 9 de octubre de 2026

**Referencia del repositorio:** `c3a0358`

**Implementación:** C# · .NET 10 · Windows x64

**Destinatarios:** desarrollo, mantenimiento y revisión técnica.

> Esta guía describe la implementación C# presente en el repositorio. Explica las decisiones observables y las garantías implementadas, e identifica los puntos de extensión para mantener la herramienta.

Fuentes: código de `src/`, pruebas de `tests/`, configuración de proyectos, README y reglas de AGENTS.md. Los ejemplos de personas y archivos son ficticios.

MAPA DE LECTURA

# Índice


| Contenido | Pág. |
|---|---|
| 1. Propósito, alcance y organización | 3 |
| 2. Arquitectura y composición de dependencias | 4 |
| 3. Punto de entrada y recorrido de la aplicación | 5 |
| 4. CLI, asistente y contratos de ejecución | 6 |
| 5. Identidad e interpretación de nombres | 7 |
| 6. Flujo de ordenación simple | 8 |
| 7. Lectura estructurada de DOCX | 9 |
| 8. Extracción y validación del listado PDF | 10 |
| 9. Planificación de correspondencias | 11 |
| 10. Flujo transaccional de transferencia | 12 |
| 11. Importación de formato y recursos | 13 |
| 12. Validación del resultado y límites actuales | 14 |
| 13. Compilación, pruebas y mantenimiento | 15 |
| 14. Navegación del código y diagnóstico | 16 |


## Lectura recomendada

Para entender una ejecución completa, comienza por los apartados 2–4 y continúa por el 6 (ordenación) o por los apartados 7–12 (transferencia). Para modificar reglas de negocio, revisa primero identidad, ordenación y planificación.

## Convenciones

- Las rutas son relativas a la raíz del repositorio.
- Los nombres de clases y métodos se conservan en inglés, tal como aparecen en el código.
- «Origen» aporta las filas; «destino» aporta la base del documento final; «listado» determina las identidades solicitadas.
- La conservación de una parte del DOCX se refiere a sus bytes descomprimidos, no a la identidad binaria del ZIP completo.

La revisión realizada para elaborar este documento es estática y documental. Las cifras históricas de certificación no representan una nueva ejecución de las pruebas.

# Propósito, alcance y organización

DocMerge es una aplicación de consola que modifica tablas de documentos Word mediante sus partes XML. Su unidad de trabajo es una fila completa `<w:tr>`; el texto extraído sirve para identificar y ordenar, pero no para reconstruir el documento.

## Dos operaciones principales

| Ordenación simple | Transferencia por listado |
|---|---|
| Recibe un DOCX, localiza una cabecera única y reordena sus filas de personas. | Recibe origen DOCX, destino DOCX y listado PDF. Calcula acciones de añadir, reemplazar, conservar, eliminar u omitir. |
| El resultado toma como base el propio documento de entrada. | El resultado toma como base el destino y añade los recursos necesarios del origen. |
| Solo cambia el contenido de `word/document.xml`. | Puede cambiar también estilos, numeración, relaciones y tipos de contenido, y añadir partes relacionadas. |

## Organización del repositorio

| Ruta | Responsabilidad |
|---|---|
| `src/` | Cuatro proyectos C#: Domain, Application, Infrastructure y Console. |
| `tests/` | Cuatro proyectos xUnit alineados con las capas. |
| `docs/` | Referencias de migración y glosario. |
| `legacy/python/` | Oráculo histórico inmutable para paridad; no es el motor activo. |
| `assets/documents/` `assets/outputs/` | Entradas privadas locales y resultados conservados; ignorados por Git. |
| `release/` | Entrega certificada: `DocMerge.exe` y `REEDME.txt`. |

## Plataforma y dependencias

Todos los proyectos productivos apuntan a `net10.0`. El SDK declarado es `10.0.401`, con `rollForward: latestPatch`. Infrastructure utiliza **PdfPig 0.1.16**; Console utiliza **Microsoft.Extensions.DependencyInjection 10.0.12**. DOCX se procesa con `System.IO.Compression` y `System.Xml.Linq`, sin automatizar Microsoft Word.

Referencias: `DocMerge.sln`, `global.json`, `Directory.Build.props`, `src/*/*.csproj`, `AGENTS.md`.

# Arquitectura y composición de dependencias

```
DocMerge.Console ──→ DocMerge.Application ──→ DocMerge.Domain
        │                       ↑
        └──→ DocMerge.Infrastructure ──→ DocMerge.Domain
```

**Domain** contiene identidad, interpretación de nombres, claves de ordenación, modelos documentales y planificador. No referencia otros proyectos ni bibliotecas externas. **Application** coordina casos de uso y define los contratos de acceso a documentos y archivos. **Infrastructure** implementa esos contratos. **Console** procesa argumentos, pregunta al usuario y presenta los resultados.

## Registro de servicios

`Program.BuildServices()` crea un `ServiceCollection`. Todos los registros actuales son singleton dentro del proveedor de esa ejecución.

| Contrato | Implementación registrada |
|---|---|
| `IConsoleIO` | `SystemConsoleIO` |
| `IPdfNameReader` | `PdfPigNameReader` |
| `IDocumentReader` | `DocxDocumentReader` |
| `IDocumentEditor` | `DocxDocumentEditor` |
| `IFileStore` | `PhysicalFileStore` |
| `ITransferResultValidator` | `DocxTransferValidator` |

También se registran `CliParser`, `PdfPageParser`, `TransferPlanner`, `TransferService`, `SortDocumentService` y `ConsoleApplication`. Los importadores de recursos se crean durante la operación; sus mapas de identificadores no son estado global compartido.

## Separación de interfaz y núcleo

Los casos de uso devuelven datos y excepciones; no imprimen mensajes. `TransferPlanFormatter` convierte el plan y las incidencias en texto español. Esto permite reutilizar los mismos casos de uso desde una futura interfaz WPF/MVVM, que aún no está implementada.

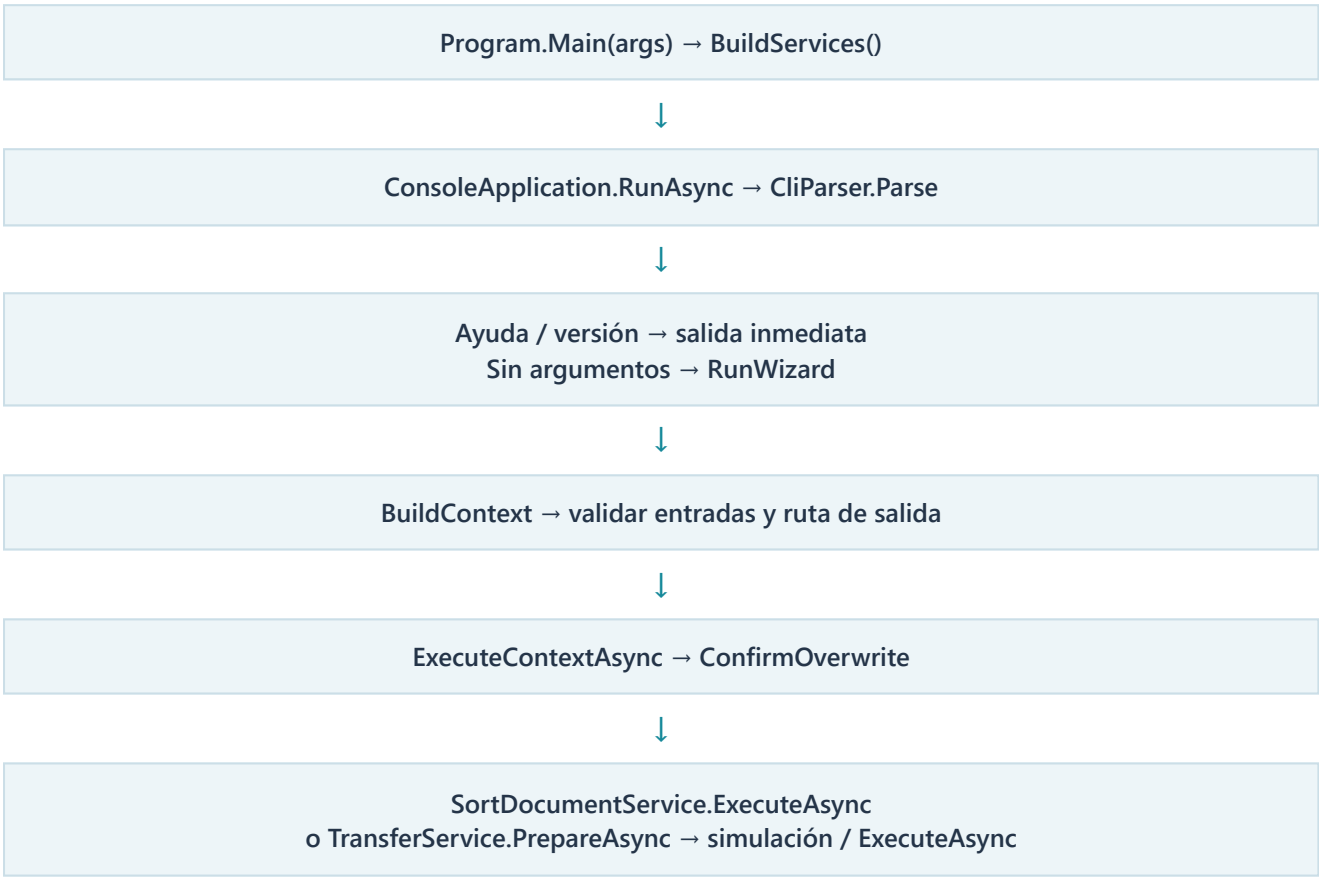
> La inversión de dependencias se realiza mediante interfaces definidas en Application. Hay acceso directo a rutas/directorios en algunos coordinadores actuales; la separación de capas no implica que toda llamada a `System.IO` esté encapsulada en `IFileStore`.

---

Referencias: `src/DocMerge.Console/Program.cs`; interfaces en `src/DocMerge.Application/{Documents,Files,Pdf,Validation}/`.

# Punto de entrada y recorrido de la aplicación

El punto de entrada es `Program.Main(string[] args)`, un método `async Task<int>`. Configura entrada y salida de consola en UTF-8, crea el proveedor de servicios con `await using`, resuelve `ConsoleApplication` y devuelve el resultado de `RunAsync`.




## Decisiones del recorrido

1. `ShowInformationalOutput` atiende ayuda y versión antes de exigir un documento. Si se piden ambas, la ayuda tiene prioridad.
2. Se considera ejecución por doble clic cualquier invocación **sin argumentos**; no se detecta el origen real del proceso.
3. `BuildContext` comprueba existencia y extensión de entradas, determina la operación y genera la salida predeterminada.
4. La salida debe diferir de todas las entradas por ruta absoluta comparada sin distinguir mayúsculas. No es una comprobación de identidad física mediante enlaces del sistema de archivos.
5. En modo sin argumentos, un `finally` solicita Intro para cerrar, incluso tras errores gestionados.

---

Referencias: `src/DocMerge.Console/Program.cs`; `src/DocMerge.Console/Cli/ConsoleApplication.cs`: `RunAsync`, `BuildContext`, `ExecuteContextAsync`.

# CLI, asistente y contratos de ejecución

| Argumento | Semántica |
|---|---|
| `ORIGEN.docx` | Único argumento posicional: documento de entrada. |
| `--destino` / `--lista` | Deben aparecer conjuntamente para transferir. |
| `-o` / `--output` | Ruta del DOCX de salida. |
| `--simular` | Prepara e imprime el plan, sin generar salida. Solo transferencia. |
| `--orden` | Identificador de uno de los perfiles del apartado 5. |
| `--cabecera` | Cabecera buscada. Predeterminada: `Nombre y Apellidos`. |
| `--force` | Autoriza sobrescribir la salida; no autoriza usar una entrada como salida. |
| `-h` / `--help`<br>`--version` | Ayuda o versión del producto. |

El parser admite `--opcion=valor`, aliases documentados y `--` para tratar el resto como posicional. En opciones repetidas prevalece el último valor. No admite abreviaciones accidentales de argparse. La selección efectiva del perfil se realiza al ejecutar, no al parsear; la simulación no llega a resolverlo.

**Salidas predeterminadas:** `ORIGEN_ordenado.docx` para ordenación y `DESTINO_generado.docx` para transferencia, junto al documento base. Si ya existe la salida, el CLI exige `--force`; el asistente pide confirmación. La simulación evita esa confirmación.

## Asistente sin argumentos

Ofrece transferencia (opción 1, predeterminada) u ordenación (opción 2). Recoge rutas y salida opcional; en transferencia pregunta si se desea simular. `CleanPath` elimina espacios externos y comillas simples/dobles de rutas arrastradas. Las respuestas afirmativas son `s`, `si` y `sí`, sin distinguir mayúsculas.

## Códigos de salida gestionados

**0:** éxito, ayuda, versión o cancelación de sobrescritura; también errores gestionados en el asistente. **1:** error operativo capturado en CLI. **2:** error de uso del CLI. No hay un código específico de incidencias de correspondencia: se comunican en el resumen.

---

Referencias: `src/DocMerge.Console/Cli/{CliParser,CliOptions,ConsoleApplication,ConsoleText}.cs`; `Presentation/TransferPlanFormatter.cs`.

# Identidad e interpretación de nombres

## Identidad: coincidencia exacta normalizada

```
IdentityNormalizer.Normalize(nombre)
    → Unicode FormC (NFC)
    → colapsar espacios Unicode y recortar extremos
    → full case folding Unicode 15.0
    → clave comparada con StringComparer.Ordinal
```

Las tablas de espacios y case folding están embebidas en Domain. Se recorren valores Unicode mediante `Rune`, no unidades UTF-16 aisladas. No se eliminan tildes ni la ñ; tampoco hay coincidencia aproximada, inversión automática de palabras o búsqueda fonética.

Por ejemplo, « `García López Ana` » coincide con «`GARCÍA LÓPEZ ANA`», pero no con «`Garcia Lopez Ana`». El full case folding también maneja expansiones como `ß → ss`, que un simple `ToLowerInvariant` no reproduce.

## Interpretación histórica para ordenar

`LegacyNameInterpreter.Interpret` colapsa espacios y divide por palabras. Con tres o más palabras, asigna **todas salvo las dos últimas** al primer apellido, la penúltima al segundo apellido y la última al nombre. Es una heurística de compatibilidad: no interpreta semánticamente nombres compuestos.

| Perfil público | Tupla de comparación |
|---|---|
| `apellido-nombre-segundo-apellido` (predeterminado) | Primer apellido, nombre, segundo apellido. |
| `nombre-apellidos` | Nombre, primer apellido, segundo apellido. |

`SortingProfile.CreateKey` aplica case folding a cada componente. `SortKeyComparer` compara la tupla lexicográficamente mediante `PythonStringComparer`, por puntos de código Unicode, no por orden cultural español. La construcción de la clave no repite la normalización NFC de identidad.

> Con menos de tres palabras se obtiene `SortKey.Invalid`, una tupla de U+FFFF. Queda tras nombres habituales; no constituye un máximo absoluto frente a todos los caracteres Unicode suplementarios. `OrderBy` conserva el orden relativo de claves iguales.

---

Referencias: `src/DocMerge.Domain/Identity/`, `Unicode/`, `Sorting/{LegacyNameInterpreter,SortingProfile,SortingProfiles,SortKeyComparer,PythonStringComparer}.cs`.

06 · CASO DE USO

# Flujo de ordenación simple

```
ConsoleApplication.ExecuteContextAsync
  → SortDocumentService.ExecuteAsync
    → IDocumentEditor.SortAsync
      → DocxDocumentEditor.SortAsync
        → DocxPackage.FromBytes
        → DocxXml.LocateStrictTable
        → ReorderRows → DocxPackage.WriteAsync
```

1. **Leer:** carga el DOCX completo de forma asíncrona. `DocxPackage` descomprime sus entradas en un diccionario de bytes y conserva la secuencia original de partes.
2. **Localizar:** analiza `word/document.xml` preservando espacios XML. Busca una única celda cuyo texto normalizado coincida exactamente con la cabecera. De ella obtiene tabla, fila de cabecera y columna de identidad.
3. **Seleccionar:** examina las filas posteriores a la cabecera. Ignora las que no alcanzan la columna de identidad o tienen su texto vacío tras `Trim()`.
4. **Ordenar:** calcula claves del perfil, clona cada `w:tr` y reemplaza únicamente las posiciones de las filas seleccionadas. Las filas auxiliares quedan en su posición.
5. **Escribir:** serializa solo `word/document.xml`, genera un ZIP temporal en el directorio de salida y lo mueve a la ruta final.
6. **Informar:** devuelve el número de filas ordenadas. Console imprime ruta de salida y recuento.

## Qué conserva

Al mover filas completas mantiene su contenido y formato explícito, incluidos párrafos, propiedades de celda y referencias internas. Las demás partes conservan sus bytes descomprimidos. La recomposición del ZIP puede cambiar compresión o metadatos del contenedor; no se promete igualdad binaria del archivo completo.

## Diferencias respecto a la transferencia

La ordenación no usa `DocxDocumentReader` ni `DocxTransferValidator`. Exige cabecera exacta única y al menos una fila ordenable, pero no aplica el perfil estricto de tres celdas ni todas las restricciones estructurales del lector de transferencia. Tampoco realiza la comprobación de hashes de tres entradas.

La publicación temporal protege frente a una escritura incompleta; no existe una validación semántica posterior equivalente a la transferencia. La corrección aprobada exige derivar la columna de la cabecera: no se presupone la segunda columna ni se aceptan coincidencias parciales.

---

Referencias: `src/DocMerge.Application/Sorting/SortDocumentService.cs`; `src/DocMerge.Infrastructure/Docx/{DocxDocumentEditor,DocxXml,DocxPackage}.cs`.

# Lectura estructurada de DOCX

`DocxDocumentReader.ReadAsync` valida la ruta, lee los bytes y delega en `Parse`. Produce una instantánea del documento, junto con el hash SHA-256 calculado sobre esos mismos bytes.

## Contrato de tabla de transferencia

- Una única coincidencia exacta normalizada de la cabecera entre las celdas de las tablas de `word/document.xml`.
- Cabecera y cada fila posterior con exactamente **tres celdas directas**. La columna de nombre puede ser cualquiera de ellas.
- Al menos un registro de persona. Una fila con todas sus celdas textualmente vacías se conserva como auxiliar.
- Si una fila contiene texto pero no tiene identidad en la columna derivada de la cabecera, se rechaza.
- En la tabla objetivo se rechazan combinaciones verticales/horizontales, tablas anidadas, inserciones/eliminaciones/movimientos de control de cambios, referencias a comentarios y notas al pie/finales.

## Modelos que cruzan las capas

| Modelo | Contenido relevante |
|---|---|
| `DocumentSnapshot` | Ruta, SHA-256, índices de tabla/cabecera/columna, registros y filas auxiliares. Expone agrupaciones duplicadas. |
| `DocumentRecord` | Nombre original, clave de identidad, ubicación, XML de la fila completa y dependencias. |
| `RecordDependencies` | Identificadores de estilos, numeración y relaciones usadas por la fila. |
| `RelationshipReference` | ID, tipo, destino y marca de relación externa. |
| `AuxiliaryRow` | Índice y XML de una fila textualmente vacía. |

Los índices de tabla y fila proceden de enumeraciones con base cero. Las ubicaciones del PDF, en cambio, utilizan página y ordinal con base uno.

## Resolución de recursos

El lector examina los atributos de relación de las filas, exige que cada ID esté declarado y que los destinos internos existan en el paquete. Las relaciones externas se conservan como referencias, sin descargarlas. `PackagePaths` resuelve rutas relativas y rechaza recorridos que salgan del paquete.

---

Referencias: `src/DocMerge.Infrastructure/Docx/DocxDocumentReader.cs`, `DocxXml.cs`, `PackagePaths.cs`; modelos en `src/DocMerge.Domain/Documents/`.

# Extracción y validación del listado PDF

`PdfPigNameReader` implementa `IPdfNameReader`. Comprueba extensión y existencia, lee los bytes, abre el documento con PdfPig y extrae cada página mediante `ContentOrderTextExtractor.GetText`. Comprueba cancelación entre páginas.

## Normalización previa

`CanonicalizePage` colapsa espacios por línea y separa dos formas que el extractor puede devolver juntas: la cabecera de número/nombre y una entrada con número seguido de texto. Así, `1 García López Ana` se transforma en una línea de ordinal y otra de nombre.

## Parser independiente de la biblioteca

`PdfPageParser.Parse` recibe una lista de textos de página; sus reglas se pueden probar sin abrir un PDF real.

1. Rechaza ausencia de páginas o texto completamente vacío.
2. Normaliza saltos de línea y espacios. Busca secciones `Alumnado (n)` y comprueba que todos sus totales coincidan y sean positivos.
3. Exige una cabecera `Nombre y apellidos` después de una sección. En páginas de continuación admite repetición de cabecera o continuación desde el inicio si ya empezó el listado.
4. Localiza ordinales; une las líneas de cada nombre hasta el siguiente ordinal dentro de esa página, excluyendo líneas de control conocidas.
5. Exige que el número de candidatos sea exactamente `n` y que los ordinales sean `1…n`, en ese orden y sin saltos.
6. Devuelve `PdfReadResult` con entradas, total declarado y páginas. El lector añade el SHA-256 del archivo leído.

## Trazabilidad y errores

Cada `PdfEntry` conserva texto original, identidad normalizada, página y ordinal. Los duplicados normalizados no se eliminan aquí: el planificador los agrupa y genera una incidencia conservando las referencias.

> No hay OCR ni diálogo de contraseña. Un PDF sin texto extraíble, inaccesible por cifrado o incompatible con el formato esperado provoca un error. Las comprobaciones de total y numeración detectan muchas extracciones parciales; no verifican visualmente que todo el texto extraído sea correcto.

---

Referencias: `src/DocMerge.Infrastructure/Pdf/{PdfPigNameReader,PdfPageParser}.cs`; `src/DocMerge.Domain/Pdf/`; excepciones en `src/DocMerge.Application/Pdf/`.

# Planificación de correspondencias

`TransferService.PrepareAsync` lee, en este orden, PDF, origen y destino. Entrega sus instantáneas a `TransferPlanner.Create`, un componente de dominio sin E/S. El resultado es un `TransferPlan` con acciones, incidencias, hashes de las tres entradas y total declarado.

| Situación | Acción | Resultado |
|---|---|---|
| Identidad listada, un origen, sin destino. | `Add` | Incorporar fila del origen. |
| Identidad listada, un origen y un destino. | `Replace` | Sustituir fila completa del destino. |
| Identidad listada, origen ausente o múltiple, un destino. | `Keep` | Conservar destino e informar incidencia. |
| Identidad listada, origen ausente o múltiple, sin destino. | `Skip` | No crear fila; informar incidencia. |
| Identidad duplicada en destino, esté o no en el PDF. | `Keep` por fila | Conservar todas las filas; incidencia de destino ambiguo. |
| Identidad única en destino y ausente del PDF. | `Delete` | Eliminar esa fila del resultado. |

## Orden de las decisiones

Primero agrupa por identidad con comparación ordinal. Registra duplicados del PDF; detecta destinos ambiguos y los conserva; resuelve las identidades del listado; finalmente añade eliminaciones de destinos no listados y no ambiguos. Las incidencias de destino ambiguo siguen la primera aparición en el documento de destino.

Una identidad repetida en el PDF genera `DuplicatePdfEntry` pero se resuelve una vez como grupo; por sí sola no impide transferir si el origen es único. Las demás incidencias son `MissingSource`, `AmbiguousSource` y `AmbiguousDestination`.

## Ejemplo mínimo

El PDF solicita «García López Ana» y «Pérez Ruiz Luis». El origen contiene una fila de Ana y ninguna de Luis; el destino contiene a Luis y a «Soto Gil Eva». El plan añade a Ana, conserva a Luis con incidencia y elimina a Eva. Posteriormente ordena las filas resultantes según el perfil.

**Importante:** el total final no tiene por qué igualar el total del PDF, debido a duplicados, omisiones y conservación de destinos ambiguos. La validación usa las filas esperadas del plan.

---

Referencias:
`src/DocMerge.Domain/Transfer/{TransferPlanner,TransferPlan,TransferAction,TransferIssue,ActionType,IssueType}.cs`.

10 · CASO DE USO

# Flujo transaccional de transferencia

```
PrepareAsync → TransferPlan
   ├─ --simular → imprimir acciones e incidencias → fin
   └─ ExecuteAsync
        → temporal «transferido»: ApplyTransferPlanAsync
        → temporal «ordenado»: SortAsync
        → VerifyInputsAsync: hashes de origen/destino/PDF
        → ITransferResultValidator.ValidateAsync
        → IFileStore.PublishAsync → salida final
        → finally: eliminar ambos temporales
```

`TransferExecutionRequest` agrupa entradas, salida, perfil y plan opcional. Si no recibe plan, `ExecuteAsync` lo prepara. El CLI normalmente reutiliza el plan que acaba de calcular. Los temporales se crean en el directorio final, con GUID y sufijos descriptivos.

## Aplicación del plan

`DocxTransferEditor.ApplyAsync` vuelve a leer origen y destino, verifica sus hashes frente al plan y carga ambos paquetes. Extrae del plan únicamente filas `Add`/`Replace`, importa sus dependencias y opera sobre el paquete del destino.

`ApplyActions` conserva un mapa de filas por índice original: las eliminaciones retiran nodos, las sustituciones reemplazan nodos completos y las altas se añaden al final de la tabla. `Keep` y `Skip` no alteran nodos. Después se ejecuta la ordenación del documento temporal.

## Verificación y publicación

Antes de validar y publicar se recalculan los hashes de las tres entradas. Si difieren de los incluidos en el plan, la operación falla. Un hash vacío se omite, comportamiento útil para dobles de prueba; los lectores reales sí generan hashes.

`PhysicalFileStore.PublishAsync` comprueba cancelación y ejecuta `File.Move(..., overwrite: true)`. La salida anterior no se sustituye hasta superar la validación. Los bloques `finally` intentan limpiar temporales tanto en éxito como en fallo.

## Progreso y cancelación

La API recibe `CancellationToken` y un `IProgress<OperationProgress>` opcional. Informa etapas de lectura, importación, ordenación, validación y publicación; `Planning` está declarado pero no se emite actualmente. El CLI pasa progreso nulo y `CancellationToken.None`: no incorpora un flujo cooperativo propio para Ctrl+C.

---

Referencias: `src/DocMerge.Application/Transfer/TransferService.cs`; `src/DocMerge.Infrastructure/Docx/DocxTransferEditor.cs`; `Files/PhysicalFileStore.cs`.

# Importación de formato y recursos

Copiar únicamente `w:tr` no basta si la fila referencia estilos, numeración o imágenes del origen. `ImportDependencies` ejecuta los importadores y obtiene mapas para reescribir referencias en las filas clonadas.

## 1. StyleImporter

Recoge estilos referenciados por las filas y recorre dependencias `basedOn` y `link`. Exige que existan las partes de estilos de origen y destino cuando son necesarias. Reutiliza una definición ya presente si no hay conflicto; ante un mismo ID con XML distinto genera IDs como `DO_Estilo` o `DO_Estilo_2`.

Propaga conflictos a estilos que dependen de otros conflictivos, clona las definiciones necesarias y remapea enlaces internos. No es una fusión general de toda la semántica de estilos de Word: el recorrido explícito de dependencias está acotado a esos enlaces.

## 2. NumberingImporter

Recoge `numId` de filas y estilos importados, excluyendo `0`. Requiere `word/numbering.xml` en ambos paquetes si hay numeración que importar. Asigna IDs nuevos por encima de los existentes a numeraciones y definiciones abstractas, reutiliza el mapa por definición y remapea vínculos de estilo y numeración de los estilos importados.

## 3. RelationshipImporter

Asigna IDs de relación libres (`rIdN`). Para destinos internos, importa los bytes de la parte y recorre sus relaciones dependientes. Si un nombre de parte ya existe, utiliza uno como `docmerge_1_image1.png`. Un mapa evita volver a importar la misma parte y permite resolver grafos con referencias compartidas.

Las relaciones externas conservan su destino. Se actualizan los tipos de contenido mediante declaraciones `Override` o `Default` cuando corresponda. Los recursos nuevos no sobrescriben los recursos existentes del destino.

## 4. Remapeo y escritura

`RemapSourceRow` actualiza atributos de estilo, `numId` y relaciones en una copia de la fila. El paquete final mantiene el destino como base. Las partes originales se escriben en su orden inicial y después se incluyen las nuevas.

> La conservación se apoya en XML completo y recursos explícitos. No hay renderizado de Word, comparación visual de páginas ni reconstrucción integral de temas, valores predeterminados o cualquier estructura OOXML arbitraria.

---

Referencias:
`src/DocMerge.Infrastructure/Docx/{StyleImporter,NumberingImporter,RelationshipImporter,DocxTransferEditor,PackagePaths,DocxPackage}.cs`.

# Validación del resultado y límites actuales

`DocxTransferValidator.ValidateAsync` vuelve a leer el resultado con el lector estricto antes de autorizar la publicación.

| Comprobación implementada | Mecanismo |
|---|---|
| Personas y orden esperados. | Filas de origen para Add/Replace, filas de destino para Keep; excluye Delete/Skip. Ordena lo esperado y compara cantidad e identidad por posición. |
| Contenido y formato explícito de filas. | `XNode.DeepEquals` sobre firmas de filas; sustituye IDs remapeables por un marcador común antes de comparar. |
| Partes originales del destino. | Exige que todas sigan presentes y sean iguales en bytes salvo las cinco partes autorizadas. |
| Contenido fuera de la tabla. | Retira la tabla objetivo de ambos XML y compara el resto con `XNode.DeepEquals`. |
| Referencias de filas. | El lector resuelve relaciones; el validador exige IDs de estilos y numeraciones declarados, salvo numId 0. |

Las cinco partes con cambios permitidos son `word/document.xml`, `word/styles.xml`, `word/numbering.xml`, `word/_rels/document.xml.rels` y `[Content_Types].xml`. La comprobación de partes originales permite añadir partes nuevas.

## Alcance exacto

- La comparación de firmas tolera cambios en IDs; no demuestra por sí sola equivalencia semántica de cada definición importada ni igualdad visual del documento.
- El lector identifica filas auxiliares por texto vacío. El validador no compara explícitamente todas las filas auxiliares o propiedades internas de la tabla con el destino.
- Los hashes detectan cambios en los puntos de comprobación; no bloquean los archivos durante toda la transacción ni eliminan toda posible carrera entre comprobación y publicación.
- Lecturas de archivos y escrituras usan APIs asíncronas, pero análisis XML, ZIP, ordenación y parte de la validación son trabajo síncrono en memoria. `async` no implica procesamiento íntegramente en segundo plano.

## Errores

Application define `DocMergeException` y excepciones específicas de documento, PDF y validación. Console captura errores de uso por separado y un conjunto de errores operativos (`IOException`, `InvalidDataException`, `ArgumentException`, `DocMergeException`). No es un capturador universal de excepciones.

---

Referencias: `src/DocMerge.Infrastructure/Docx/DocxTransferValidator.cs`; `src/DocMerge.Application/Errors/`; `src/DocMerge.Console/Cli/ConsoleApplication.cs`.

# Compilación, pruebas y mantenimiento

## Comandos desde la raíz del repositorio

```
dotnet format --verify-no-changes
dotnet build DocMerge.sln -warnaserror
dotnet test DocMerge.sln

dotnet run --project src/DocMerge.Console -- ORIGEN.docx
dotnet run --project src/DocMerge.Console -- ORIGEN.docx --destino DESTINO.docx --lista LISTA.pdf -o
RESULTADO.docx --simular

dotnet publish src/DocMerge.Console -c Release -r win-x64 --self-contained true -
p:PublishSingleFile=true
```

Si el comando resuelve la instalación x86 sin SDK, utilizar el ejecutable de `C:\Program Files\dotnet\dotnet.exe` (en PowerShell, con & y la ruta entre comillas).

## Cobertura del repositorio

| Proyecto de pruebas | Comportamientos revisados |
|---|---|
| Domain.Tests | Unicode, perfiles y comparación por puntos de código, casos de correspondencia compartidos con Python, orden de incidencias ambiguas. |
| Application.Tests | Un fallo de validación impide publicar la salida. |
| Infrastructure.Tests | Parser PDF, lector DOCX, rechazo de cabeceras ambiguas/combinaciones, ordenación, columna derivada de cabecera, rechazo de cabecera parcial, importación y validación. |
| Console.Tests | Aliases, opciones repetidas, separador --, rechazo de abreviaciones, versión/código de error y asistente de simulación. |

La línea base registrada en AGENTS.md certificó 42 pruebas C# el 8/10/2026. El código actual incluye regresiones posteriores de ordenación; el número vigente debe obtenerse con `dotnet test`. No se ha ejecutado esa suite para este cambio exclusivamente documental.

## Versionado y entrega

`Directory.Build.props` centraliza versión 2.1.0, versiones de ensamblado/archivo 2.1.0.0, nullable y avisos como errores. Console publica un ejecutable autónomo de archivo único para Windows x64, sin trimming. Antes de entregar otro ejecutable se deben ejecutar las verificaciones, publicar y probar los flujos afectados y el asistente.

La referencia Python permanece intacta. Los documentos reales se usan solo en verificaciones locales con hashes de control; no se incorporan a pruebas compartidas ni a la distribución.

---

Referencias: `tests/`, `Directory.Build.props`, `global.json`, `.editorconfig`, `AGENTS.md`, `README.md`.

# Navegación del código y diagnóstico

| Necesidad | Punto de entrada al código |
|---|---|
| Cambiar arranque o composición. | `Console/Program.cs` |
| Seguir el CLI o el asistente. | `Console/Cli/ConsoleApplication.cs` y `CliParser.cs` |
| Añadir un formato de nombre u orden. | `Domain/Sorting/INameInterpreter.cs` y `SortingProfiles.cs`; conservar identidad independiente. |
| Investigar una correspondencia. | `Domain/Identity/IdentityNormalizer.cs` y `Domain/Transfer/TransferPlanner.cs` |
| Seguir el caso de uso transaccional. | `Application/Transfer/TransferService.cs` |
| Localizar la tabla o una fila. | `Infrastructure/Docx/DocxXml.cs` y `DocxDocumentReader.cs` |
| Revisar pérdida de formato/recurso. | `Infrastructure/Docx/DocxTransferEditor.cs` y los tres importadores. |
| Investigar rechazo del PDF. | `Infrastructure/Pdf/PdfPigNameReader.cs` y `PdfPageParser.cs` |
| Revisar rechazo del resultado. | `Infrastructure/Docx/DocxTransferValidator.cs` |

En esta tabla, Console, Domain, Application e Infrastructure abrevian las carpetas `src/DocMerge.*`.

## Diagnóstico por síntoma

- **Cabecera no encontrada o ambigua:** revisar el texto de la celda y la opción `--cabecera`; debe haber una sola coincidencia exacta normalizada.
- **Nombre no transferido:** examinar identidad completa, tildes, duplicados y tipo de incidencia. El perfil de orden no cambia la identidad.
- **Orden inesperado:** comprobar la heurística de las últimas dos palabras y recordar que se comparan puntos de código, no reglas culturales.
- **PDF rechazado:** revisar texto extraíble, sección, cabecera, total y secuencia de ordinales.
- **Resultado no publicado:** seguir el fallo de importación, hashes o validación; comprobar permisos de salida y archivos abiertos.

## Cómo ampliar la herramienta

Una nueva regla de orden se incorpora mediante estrategia de interpretación, perfil declarativo y pruebas, sin cambiar el procesador XML. Una futura interfaz consume los casos de uso y presenta planes/incidencias por su cuenta. Cambios de formato admitido, librerías, contratos o comportamiento histórico requieren una decisión explícita y regresiones adecuadas.

---

Fin de la guía · DocMerge 2.1.0 · Revisión del código `c3a0358` · 9 de octubre de 2026.